AutoCareHub — Registro de Mejoras

> *Documento vivo. Cada vez que se añade una funcionalidad nueva se registra aquí con una breve explicación, los archivos implicados y, si aplica, los cambios en la base de datos.*

Proyecto: AutoCareHub (Angular + PHP/MySQL)

Autor: Nicolás García Moreira

Última actualización: 16/10/2026

> ***Restricción del proyecto:** la API existente no se modifica. Las funcionalidades nuevas se construyen creando **llamadas nuevas**, nunca editando las llamadas ya existentes.*

Índice

1. Agenda Inteligente
2. Agenda Semanal del taller (calendario)
3. Gestión del ciclo de vida de las citas
4. Vista «Mis Citas» del cliente
5. Generador de facturas más intuitivo
6. VeriFactu (AEAT)
7. Seguridad
8. Mejoras de octubre de 2026
9. Cambios en la base de datos
10. Despliegue automático a InfinityFree
11. Próximas mejoras propuestas

---

1. Agenda Inteligente

Qué es: sustituye el sistema antiguo de citas (horario fijo 9–18, L–V, huecos de 1 hora y capacidad implícita de 1 coche) por una agenda configurable y realista.

Qué aporta:

- **Configuración por taller**: para cada día de la semana se define si abre, su hora de apertura/cierre, un descanso opcional, la **capacidad** (número de coches que puede atender a la vez / bahías) y la **granularidad** de la rejilla de huecos.
- **Catálogo de servicios con duración**: cada servicio (p. ej. «Cambio de aceite, 30 min») tiene una duración estimada y un precio opcional.

- **Disponibilidad real**: al pedir cita, el cliente elige un servicio y el sistema

calcula los huecos realmente libres teniendo en cuenta el horario, el descanso, la duración del servicio y los **solapamientos** con otras citas, respetando la capacidad.

- **Reserva segura**: la creación de la cita valida el horario y comprueba la

capacidad dentro de una **transacción**, evitando dobles reservas.

Explicación técnica: la disponibilidad se calcula recorriendo cada día del rango, generando huecos desde la apertura hasta el cierre en pasos de `SlotMinutes`, y descartando los que pisan el descanso o ya pasaron. Para cada hueco se cuentan las citas que se solapan (`inicio < finHueco AND fin > inicioHueco`); si ese número es menor que la capacidad, el hueco está disponible.

Archivos: todo el backend nuevo vive en archivos nuevos, **sin modificar la API original** (`appointments.php` queda intacto). La reserva inteligente es la acción nueva `crear_cita` en `schedule.php`.

- Backend (nuevo): `PHP/models/Schedule.php`, `PHP/controllers/ScheduleController.php`,

`PHP/routes/schedule.php`.

- Frontend: `components/agenda-config/` (configuración del taller),

`components/make-appointment/` (reserva con servicio y huecos reales).

---

2. Agenda Semanal del taller

Qué es: una vista de **calendario semanal** para el taller, en lugar de un simple listado de citas.

Qué aporta:

- 7 columnas (Lunes–Domingo) con las citas de cada día ordenadas por hora.
- Navegación entre semanas y botón «Hoy».
- Colores por estado y leyenda.
- Acciones rápidas sobre cada cita (ver punto 3).

Archivos: `components/workshop-agenda/`.

---

3. Gestión del ciclo de vida de las citas

Qué es: permite mover una cita por sus estados:

`Pendiente` → `Confirmada` → `Finalizada`, o `Cancelada`.

Qué aporta:

- El **taller** confirma / finaliza / cancela cada cita desde la Agenda Semanal.
- Cuando una cita se **cancela, su hueco se libera automáticamente** (el cálculo de disponibilidad ignora las citas canceladas).
- Validación de permisos: el taller solo puede tocar citas de su propio taller.

Archivos: acciones nuevas `cambiar_estado` y `citas_taller` en `schedule.php` (`ScheduleController` + modelo `Schedule.php`); frontend `components/workshop-agenda/`.

---

4. Vista «Mis Citas» del cliente

Qué es: una pantalla para que el cliente vea y gestione sus propias citas.

Qué aporta:

- Lista de citas con filtro **Próximas / Todas**, mostrando taller, vehículo y servicio.
- Botón **Cancelar** en las citas futuras (libera el hueco para otros clientes).
- Colores por estado.

Archivos: `components/my-appointments/`; acciones nuevas `mis_citas` y `cancelar` en `schedule.php`.

---

5. Generador de facturas más intuitivo

Qué es: se rediseña la forma en la que el taller crea una factura, sin perder la generación del PDF con los datos.

Qué aporta:

- **Cita por desplegable**: se elige la cita de una lista (`#ID · fecha · vehículo · cliente`) en lugar de escribir el ID a mano.
- **Resumen de la cita** seleccionada (estado, vehículo, cliente, servicio).
- **Ítems desde el catálogo de servicios**: un selector autocompleta descripción y precio del servicio; se mantiene la opción de escribir el ítem manualmente.
- El **PDF** se sigue generando igual (jsPDF) con todos los datos de la factura.
- **Descarga de PDF también para el taller**: la lógica del PDF se extrajo a un servicio reutilizable (`services/invoice-pdf.service.ts`) usado tanto por el cliente (*Ver facturas*) como por el taller, que ahora tiene un botón **PDF** en cada factura de su lista. No se modificó ninguna llamada de la API: se reutiliza la existente y los datos que ya devuelve. Se añade el nombre del cliente al PDF cuando está disponible.

- **Borrador de factura automático al finalizar una cita**: en la *Agenda Semanal*,

las citas *Finalizadas* muestran un botón **Facturar** que abre el generador con la cita ya seleccionada y el **servicio realizado añadido como ítem** (descripción y precio tomados del catálogo). El taller solo revisa y confirma. No se toca la API: la precarga usa la acción nueva `citas_taller` (que ahora incluye `ServiceID`) y la navegación se hace con un parámetro `?appointmentId=` en el frontend.

- **PDF de factura rediseñado** (`services/invoice-pdf.service.ts`): se sustituye el

volcado de texto sobre una imagen de fondo por un diseño profesional —cabecera con logo, «FACTURA», nº y fecha y **pastilla de estado** con color; bloques separados de **emisor** (con NIF) y **cliente + vehículo**; **tabla** de líneas con cabecera y filas alternas; **desglose Base imponible · IVA · Total** en caja; **importes en formato de moneda español (`49,90 €`); recuadro** VeriFactu con QR**; y pie de página. Solo frontend, no toca la API.

- **Estado «Facturada» en las citas**: como la API existente, al crear una factura,

deja la cita en *Finalizada* (no hay un estado «Facturada» y no se puede modificar), ese estado se **deriva** de si la cita tiene factura asociada. Las acciones nuevas `citas_taller` y `mis_citas` devuelven el `InvoiceID` de la cita, y la interfaz muestra una etiqueta **«Facturada»** (en la *Agenda Semanal* del taller y en *Mis Citas* del cliente). En la agenda, una cita ya facturada oculta el botón *Facturar* para no duplicar facturas. No se toca la API original de facturas.

Archivos: `components/invoices-generator/`, `components/invoice-viewer/`, `services/invoice-pdf.service.ts`, `components/workshop-agenda/`, `components/my-appointments/`. El desplegable de citas y el estado «Facturada» usan la acción nueva `citas_taller` de `schedule.php` (servicio + `InvoiceID`), sin tocar la API original.

---

6. VeriFactu (AEAT)

Qué es: implementación del sistema antifraude **VeriFactu** (RD 1007/2023) para los registros de facturación, respetando la restricción de no tocar la API existente (todo en endpoints y tablas nuevos, en solo lectura sobre las facturas).

Qué aporta:

- **Datos fiscales del emisor** (NIF + razón social) configurables por taller, en la

página de *Agenda Inteligente*.

- **Registro de facturación con huella encadenada**: al descargar una factura se genera

(de forma idempotente) su registro VeriFactu con una **huella SHA-256** calculada

según el orden de campos de la AEAT, que **incorpora la huella del registro anterior**

del mismo emisor. Así la secuencia de facturas es a prueba de manipulación.

- **Código QR** de verificación de la AEAT y la marca **«VERI*FACTU»** impresos en el

PDF de la factura (tanto para el taller como para el cliente).

- **Verificación de integridad de la cadena**: el taller puede recalcular toda su

cadena de huellas y comprobar que no se ha alterado ninguna factura.

Fuera de alcance (producción): el envío en tiempo real al servicio web de la AEAT (requiere certificado digital y alta en Hacienda). Aquí se genera y encadena el registro localmente, que es la parte demostrable.

Archivos (todo nuevo): BD `PHP/migrations/2026_verifactu.sql` (tablas `VerifactuEmisor` y `VerifactuRegistros`); backend `PHP/models/Verifactu.php`, `PHP/controllers/VerifactuController.php`, `PHP/routes/verifactu.php` (acciones `obtener_emisor`, `guardar_emisor`, `generar`, `obtener`, `verificar_cadena`); frontend `services/invoice-pdf.service.ts` (QR + marca), `components/agenda-config/` (datos fiscales + verificación) y dependencia `qrcode`.

---

7. Seguridad

Qué es: primera tanda de mejoras de seguridad, respetando la restricción de no modificar la API existente (solo frontend y endpoints nuevos).

Qué aporta:

- **Guards de ruta en Angular** (`services/auth.guards.ts`): ya no se puede navegar

por URL sin haber iniciado sesión, y cada ruta está **restringida por rol** (cliente / taller / administrador). Si no hay sesión redirige a `/login`; si el rol no corresponde, a `/home`. Cualquier ruta desconocida cae en `/login`.

- **Endurecimiento de los endpoints nuevos** (`schedule.php`):
- Al reservar, se comprueba que el **vehículo pertenece al usuario** autenticado

(antes se confiaba en el `VehicleID` recibido) → responde 403 si no.

- Una cita nueva **siempre nace en estado** `Pendiente`; se ignora cualquier

`Status` enviado por el cliente.

Nota: la autorización de datos la sigue garantizando el backend (sesión PHP); los

guards protegen la navegación de la interfaz. Quedan pendientes (requieren tocar la API existente, fuera del alcance actual): credenciales de BD fuera del código, cookies de sesión `HttpOnly`/`SameSite`, CORS y tokens CSRF.

Archivos: `services/auth.guards.ts`, `app.routes.ts`; backend nuevo `ScheduleController.php` y `models/Schedule.php`.

---

8. Mejoras de octubre de 2026

Tanda de mejoras implementada en la rama `mejoras/2026-10`. Como siempre, **no se modifica ningún archivo de la API original**: lo nuevo son acciones nuevas en `schedule.php` (archivo de las mejoras), endpoints nuevos (`reviews.php`, `stats.php`, `logout.php`) y cambios en el frontend.

8.1 Días festivos y cierres puntuales

Qué aporta: el taller puede bloquear fechas concretas (un día o un rango de hasta 60 días, con motivo) desde *Agenda Inteligente → Días festivos y cierres*. Esos días desaparecen de la disponibilidad que ve el cliente y la reserva los rechaza aunque el horario semanal diga que abre. Al añadir un cierre se avisa de cuántas citas activas quedan afectadas para que el taller las mueva o cancele.

Explicación técnica: tabla nueva `WorkshopClosures` (única por taller y fecha). El cálculo de disponibilidad carga los cierres del rango y salta esos días; `crear_cita` y `reprogramar` comprueban `isClosedOn()` antes de reservar.

Archivos: acciones `listar_cierres`, `crear_cierre`, `eliminar_cierre` en `schedule.php` (`ScheduleController` + `models/Schedule.php`); frontend `components/agenda-config/`.

8.2 Reprogramar cita

Qué aporta: mover una cita a otro hueco sin cancelarla y crear otra. Lo puede hacer el **cliente** desde *Mis Citas* (la cita vuelve a *Pendiente* para que el taller la confirme) o el **taller** desde la *Agenda Semanal* (botón *Mover*, conserva el estado). Un diálogo muestra solo los días y horas libres con la duración de la cita.

Explicación técnica: acción nueva `reprogramar`. Valida permisos (dueño de la cita o su taller), estado (solo Pendiente/Confirmada), formato, que no sea en el pasado, cierres, horario/descanso y capacidad dentro de una **transacción**, contando los solapamientos **sin la propia cita** (`countOverlappingExcluding`).

Archivos: `ScheduleController::reprogramar`, `models/Schedule.php`; frontend `components/reschedule-dialog/` (componente reutilizable), `my-appointments/`,

`workshop-agenda/`.

8.3 Reseñas y valoraciones

Qué aporta: tras una cita *Finalizada* el cliente puede valorar el taller (1–5 estrellas y comentario opcional), una vez por cita. La media y el número de reseñas aparecen junto a cada taller al **pedir cita**, junto con las últimas opiniones (por privacidad solo se muestra el nombre y la inicial del apellido). El taller ve en *Estadísticas* su media, el reparto por estrellas y los últimos comentarios.

Explicación técnica: tabla nueva `WorkshopReviews` (`AppointmentID` único). El backend comprueba que la cita es del usuario y está finalizada. Endpoint nuevo `reviews.php` con las acciones `crear`, `resumen_talleres` y `listar_taller`. La acción `mis_citas` devuelve ahora `ReviewRating` para saber si una cita ya está valorada.

Archivos: `PHP/models/Review.php`, `PHP/controllers/ReviewController.php`, `PHP/routes/reviews.php`; frontend `my-appointments/`, `make-appointment/`, `statistics-viewer/`.

8.4 Historial de mantenimiento por vehículo

Qué aporta: nueva pantalla *Historial vehículos* para el cliente: elige un coche y ve una línea de tiempo con todas sus citas (taller, estado, factura e importe), un resumen (citas, servicios realizados, gasto facturado) y **exportación a CSV**.

Archivos: acción nueva `historial_vehiculo` en `schedule.php` (comprueba que el vehículo es del usuario); frontend `components/vehicle-history/` y ruta `/historialVehiculo`.

8.5 Dashboard de estadísticas ampliado

Qué aporta: en *Estadísticas* se añaden indicadores (clientes atendidos, ticket medio, pendiente de cobro, valoración media), un **gráfico de ingresos** por día, semana o mes (facturado frente a cobrado), los **servicios más demandados**, las **citas por estado**, **atajos de rango (último mes / trimestre / año) y** exportación a CSV** de los ingresos y de las facturas del periodo.

Explicación técnica: endpoint nuevo `stats.php` (acción `resumen`, solo taller) que devuelve todo en una sola llamada. El formato de agrupación sale de una lista blanca en el servidor (nunca del texto que envía el cliente). Gráficos hechos con CSS, sin librerías nuevas. El CSV usa `;`, BOM UTF-8 (Excel en español) y neutraliza fórmulas.

Archivos: `PHP/models/Stats.php`, `PHP/controllers/StatsController.php`, `PHP/routes/stats.php`; frontend `statistics-viewer/` y `services/csv.util.ts`.

8.6 Recordatorios de citas en la aplicación

Qué aporta: el cliente ve en *Inicio* sus próximas citas y en *Mis Citas* un aviso

destacado con las citas de las **próximas 48 horas** (día, hora, servicio y dirección del taller). El taller ve en *Inicio* las citas de hoy y cuántas están pendientes de confirmar. (Los recordatorios por email/SMS siguen pendientes: necesitan un servidor de correo o un proveedor de SMS.)

Archivos: `components/home/`, `components/my-appointments/`.

8.7 Seguridad

- **Cerrar sesión de verdad:** botón *Cerrar sesión* en el menú que llama al endpoint nuevo `logout.php`, que destruye la sesión PHP y sus cookies (antes no había forma de cerrar sesión).
- **Protección CSRF en los endpoints nuevos y de las mejoras** (`schedule.php`, `verifactu.php`, `reviews.php`, `stats.php`, `logout.php`): solo aceptan POST con `Content-Type: application/json`, lo que obliga al navegador a hacer una petición *preflight* de CORS que solo superan los orígenes permitidos (`config/security.php`).
- **Cabeceras de seguridad** en esos endpoints: `X-Content-Type-Options`, `X-Frame-Options`, `Referrer-Policy` y `Cache-Control: no-store`.
- `PHP/.htaccess`**:** bloquea el acceso por URL a `config/`, `models/`, `controllers/`, `migrations/` y `logs/` (los logs, con emails e IDs, eran accesibles desde el navegador) y a ficheros `.log`, `.sql`, `.md`, etc. Desactiva el listado de carpetas.
- **Validación estricta de fechas** (`Y-m-d`) y rango máximo de 90 días en la disponibilidad, para evitar errores y respuestas enormes.
- **Sesión caducada:** un interceptor HTTP detecta las respuestas 401, limpia los datos locales y lleva al login con el aviso «Tu sesión ha caducado».
- Las pantallas de login y registro redirigen a *Inicio* si ya hay sesión.

8.8 Modo oscuro, accesibilidad y experiencia de uso

- **Modo oscuro** con botón en el menú; recuerda la preferencia y, si no hay ninguna, sigue la del sistema operativo. Se aplica invirtiendo la luminosidad de la página (las imágenes se vuelven a invertir), así funciona en todas las pantallas sin reescribir sus estilos.
- **Accesibilidad:** enlace «Saltar al contenido», foco visible al navegar con teclado, `aria-label`/roles en menú y diálogos, enlace activo marcado (`aria-current`), `lang="es"` en el documento, mensajes de error con `role="alert"`, respeto a `prefers-reduced-motion` y botón para **mostrar/ocultar la contraseña** en el login.

- **Menú renovado:** marca de la app, enlace activo resaltado, nombre del usuario y

menú plegable en móvil (botón hamburguesa).

- **Idioma recordado** entre visitas (antes volvía siempre a español).
- **Carga diferida de pantallas** (`loadComponent`): el bundle inicial de producción baja

de **1,26 MB a 0,71 MB** (139 kB comprimido); librerías pesadas como jsPDF solo se descargan al abrir una pantalla que las usa.

- Los estilos globales (`styles.css`) se cargan **después** de Bootstrap para que puedan

sobrescribirlo.

8.9 Taller en 3D

Qué es: una vista tridimensional del taller (menú *Taller 3D*, solo rol Taller) en la que se ven los elevadores, el foso, las plazas y la zona de lavado, con los coches que hay dentro en ese momento.

Qué aporta:

- **Coches en su puesto:** el panel lateral lista los coches con cita ese día («Por

colocar»); con un clic se meten en un puesto (el primero libre o el que se elija). Cada coche muestra su matrícula y modelo, un anillo con el color del estado de la cita y su propia matrícula en el parachoques.

- **Ficha del coche al pulsarlo:** matrícula, marca, modelo y año, cliente y email,

servicio, horario, estado y factura; acciones *Confirmar / Finalizar / Facturar*; mover a otro puesto, **subir o bajar el elevador** (con animación), cambiar el color del coche y sacarlo del taller; **notas del vehículo** (se guardan y se ven en futuras visitas del mismo coche) y **visitas anteriores** a este taller.

- **Totalmente personalizable** (modo *Editar distribución*): añadir puestos de 5 tipos

(elevador de 2 columnas, elevador de 4 columnas, foso, plaza/espera y lavado), **moverlos arrastrando** sobre el suelo (rejilla de 0,25 m), girarlos, duplicarlos, borrarlos, ponerles nombre y color propio; cambiar el **tamaño de la nave**, los colores de suelo, paredes, elevadores y líneas, y el **cartel** con el nombre del taller. Se puede descartar o restablecer la distribución por defecto.

- Cámara orbital (girar, acercar, desplazar) con botones de *vista inicial* y *vista de

planta*. Selector de día para preparar el taller de otro día.

Explicación técnica: escena hecha con **Three.js** (`garage-scene.ts`, sin modelos externos: coches, elevadores y rótulos se generan por código, con sombras y texturas

dibujadas en canvas). El bucle de render corre fuera de la detección de cambios de Angular y la pantalla se carga de forma diferida, así que Three.js solo se descarga al abrir *Taller 3D*. El servidor **valida y sanea** la distribución antes de guardarla (tipos permitidos, máximo 40 puestos, claves únicas, coordenadas dentro de la nave, ángulos normalizados, colores `#RRGGBB`, textos sin HTML). Si se borra un puesto con coche, ese coche vuelve a «Por colocar».

Archivos: backend nuevo `PHP/models/Garage.php`, `PHP/controllers/GarageController.php`, `PHP/routes/garage.php` (acciones `obtener`, `guardar_layout`, `restablecer_layout`, `asignar`, `actualizar_coche`, `retirar`, `detalle`, `crear_nota`, `eliminar_nota`); frontend `components/workshop-3d/` y dependencia `three`.

8.10 Base de datos de estas mejoras

Migración nueva `PHP/migrations/2026_10_mejoras.sql` (aditiva, no toca tablas existentes). Además, `PHP/config/SchemaUpgrade.php` crea estas tablas automáticamente la primera vez que se usan las llamadas nuevas, por lo que la migración manual es opcional.

Cambio	Descripción
Tabla `WorkshopClosures`	Días de cierre por taller: fecha y motivo (única por taller y día).
Tabla `WorkshopReviews`	Reseñas: cita (única), cliente, taller, estrellas 1–5 y comentario.
Tabla `WorkshopLayouts`	Distribución del taller en 3D (JSON validado en el servidor).
Tabla `WorkshopBayAssignments`	Qué cita/coche ocupa cada puesto, su color y si está elevado.
Tabla `VehicleNotes`	Notas del taller sobre cada vehículo.

---

9. Cambios en la base de datos

Migración: `PHP/migrations/2026_agenda_inteligente.sql` (idempotente y aditiva; no borra datos). Resumen:

Cambio	Descripción
Tabla `WorkshopSchedules`	Horario semanal por taller: día, abierto/cerrado, apertura, cierre, descanso, capacidad y granularidad.
Tabla `WorkshopServices`	Catálogo de servicios por taller: nombre, duración, precio, activo.
Columna `Appointments.ServiceID`	Enlaza cada cita con el servicio elegido (FK opcional).
Índice `idx_appt_workshop_time`	Acelera las consultas de disponibilidad.

Cambio	Descripción
Tabla `WorkshopClosures`	Días festivos / cierres puntuales (ver 8.10, `2026_10_mejoras.sql`).
Tabla `WorkshopReviews`	Reseñas y valoraciones (ver 8.10, `2026_10_mejoras.sql`).
Tablas `WorkshopLayouts`, `WorkshopBayAssignments`, `VehicleNotes`	Taller en 3D (ver 8.9 y 8.10).

También se actualizó `PHP/paginaCreacionBase.php` para que una base de datos nueva incluya ya estas tablas y columnas.

---

10. Despliegue automático a InfinityFree

Qué es: cada `push` a la rama `main` publica la web en InfinityFree sin pasos manuales, mediante GitHub Actions.

Cómo funciona (`.github/workflows/deploy-infinityfree.yml`):

1. Compila el frontend Angular en modo producción (usa `environment.prod.ts`).
2. Sube `AUTOCAREHUB/dist/autocarehub/browser/` por FTP a `/htdocs/Autocare/frontend/`.
3. Sube la carpeta `PHP/` a `/htdocs/Autocare/BACKEND/`, **excepto** `config/database.php` (en el servidor tiene los datos de la BD de InfinityFree) y los logs.

Solo se suben los archivos que han cambiado desde el último despliegue. También se puede lanzar a mano desde la pestaña *Actions* de GitHub.

Configuración necesaria: en GitHub › *Settings › Secrets and variables › Actions*, los secretos `FTP_SERVER`, `FTP_USERNAME` y `FTP_PASSWORD` (datos FTP del panel de InfinityFree).

Otros cambios: el `baseHref` de `angular.json` pasa a `/Autocare/frontend/` (en minúsculas, como la carpeta del servidor).

Nota: las migraciones SQL no se ejecutan solas; hay que lanzarlas a mano en la BD de producción antes de desplegar código que las necesite.

---

11. Próximas mejoras propuestas

Ideas que encajan con lo construido, ordenadas por relación valor / esfuerzo:

Agenda

- **Lista de espera** cuando un hueco está completo (avisar al liberarse por cancelación o reprogramación).

Comunicación

- **Recordatorios automáticos por email/SMS** (los recordatorios dentro de la app ya

existen; falta el envío externo, que requiere SMTP o un proveedor de SMS. La tabla `NotificationPreferences` ya existe en el modelo de datos).

- Que el taller pueda **responder a las reseñas**.
- **Taller 3D:** que el cliente vea en qué puesto está su coche y su estado en tiempo

real, y adjuntar fotos a las notas del vehículo.

Cliente / negocio

- **Pago online** de facturas (Stripe en modo test) y marcado automático como *Pagada*.
- **Buscador de talleres por cercanía** con mapa (requiere guardar coordenadas).

Calidad y seguridad

- **Recuperar contraseña** por email (requiere SMTP).
- Sacar las **credenciales de BD** del código, cookies de sesión `HttpOnly`/`SameSite`

y protección CSRF en los endpoints **originales** (requieren tocar la API existente).

- **Tests automáticos** de los endpoints nuevos y de los componentes (las pruebas

`*.spec.ts` actuales son las generadas por defecto).

---

*Fin del documento. Añadir las nuevas mejoras al final de la sección correspondiente

y actualizar la fecha de la cabecera.*